

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Arhitectură Agentică pentru Asistență Inteligentă în
Dezvoltarea Software**

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare si Tehnologia Informatiei*
programul de studii *Ingineria Informatiei***

Conducător științific
Ș.L. dr. ing. Mihai DOGARIU

Absolvent
Eduard-Daniel NANESCU

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **Eduard-Daniel NANESCU, 441A**

1. Titlul temei: Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software

2. Descrierea temei:

Contribuția originală a lucrării va consta în dezvoltarea unei arhitecturi software bazată pe Modele Mari de Limbaj (LLMs - Large Language Models), capabilă să asiste utilizatorul în scrierea, depanarea, documentarea și explicarea codului sursă. Aplicațiile vizate sunt eficientizarea procesului de dezvoltare software și automatizarea sarcinilor repetitive întâlnite în programare. Sistemul propus nu vizează antrenarea de la zero a modelelor, ci implementarea unei soluții agentice care utilizează modele pre-antrenate pentru a executa acțiuni complexe. Cercetarea aferentă va include: sinteza bibliografică a realizărilor actuale din domeniu, utilizarea unui mecanism care permite modelului să utilizeze instrumente pentru a interacționa cu mediul virtual, implementarea unor tehnici de inginerie a prompturilor și management-ul contextului pentru a asigura relevanța răspunsurilor, validarea experimentală a prototipului dezvoltat prin scenarii de utilizare reale utilizând medii de dezvoltare precum PyCharm/VS Code și biblioteci specifice precum LangChain, LangGraph sau GoogleADK, concluzionarea rezultatelor obținute și a contribuției originale, cât și enunțarea perspectivelor ulterioare de cercetare.

3. Contribuția originală:

Proiectarea și implementarea arhitecturii agentului, incluzând logica de selecție și utilizare a instrumentelor, gestionarea contextului și dezvoltarea unei interfețe sau integrarea cu interfețe existente care să permită interacțiunea facilă cu asistentul.

4. Resurse și bibliografie:

- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. "React: Synergizing reasoning and acting in language models." In The eleventh international conference on learning representations. 2022.
- Bozkurt, Aras, and Ramesh C. Sharma. "Generative AI and prompt engineering: The art of whispering to let the genie out of the algorithmic world." Asian Journal of Distance Education 18, no. 2 (2023): i-vii.
- Alto, Valentina. Building LLM Powered Applications: Create intelligent apps and agents with large language models. Packt Publishing Ltd, 2024.
- Zhu, He, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao et al. "Oagents: An empirical study of building effective agents." arXiv preprint arXiv:2506.15741 (2025).

5. Data înregistrării temei: 2025-12-11 22:54:00

Conducător(i) lucrare,
Ș.L. dr. ing. Mihai DOGARIU

Student,
Eduard-Daniel NANESCU

Director departament,
Conf. dr. ing Bogdan FLOREA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **621d9dfde3**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Eduard-Daniel NANESCU

.....

Cuprins

LISTA FIGURILOR	iii
LISTA TABELELOR	iv
LISTA DE ACRONIME	v
INTRODUCERE	1
CONTEXT SI MOTIVATIE	1
SCOPUL SI OBIECTIVELE LUCRARI	1
1. LUCRARI ANTERIOARE	3
1.1. Drumul catre modelele mari de limbaj (Large Language Model (LLM))	3
1.2. Maturizarea modelelor mari de limbaj si cercetari actuale	5
1.3. Solutii agentice pentru industria dezvoltarii software	6
1.4. Necesitati pentru dezvoltarea unei solutii agentice	8
1.5. Puncte slabe ale arhitecturilor agentice	8
1.6. Alegerea solutiei de implementare pe baza lucrarilor studiate	8
2. IMPLEMENTARE	9
2.1. Inceput	9
2.2. Proces	9
2.3. sys prompt	9
2.4. Provocari	9
2.5. Solutii	9
BIBLIOGRAFIE	10

LISTA FIGURILOR

1.1.	Modelul unui perceptron	3
1.2.	Convolutia cu un nucleu a unei imagini	4
1.3.	Recurrent Neural Network (RNN) si Long Short Term Memory (LSTM) comparate	4
1.4.	Arhitectura transformer	5
1.5.	Evolutia modelelor in anii recenti [1]	6
1.6.	Evolutia modelelor GPT [1]	6
1.7.	Clasamentul modelelor in SWE-Bench Pro [2] la momentul actual in functie de performanta rezolvarii sarcinilor complexe in limba engleza	8
1.8.	Clasamentul modelelor in SWE-Bench Pro [2] la momentul actual in functie de cost	8

LISTA TABELELOR

1.1. Comparatia modelelor existente si a capabilitatilor acestora	7
---	---

LISTA DE ACRONIME

API Application Programming Interface

BTT Backpropagation Through Time

CD Continuous Deployment

CI Continuous Integration

CNN Convolutional Neural Network

E2E End to End

IA Inteligență Artificială

LLM Large Language Model

LSTM Long Short Term Memory

OOS Out of Scope

RCA Root Cause Analysis

RNN Recurrent Neural Network

SLM Small Language Model

TUI Terminal User Interface

INTRODUCERE

CONTEXT SI MOTIVATIE

Motivatia acestei lucrari vine din experienta personala in industria dezvoltarii software. Atributiile unui inginer software sunt multe, diverse si adesea repetitive si ingreuneaza procesul de dezvoltare. A scrie cod a devenit doar o mica parte din sarcinile intrebuintate unui inginer software, pierzand teren in fata design-urilor, documentatiei, review-urilor, a testelor unitare de integrare si End to End (E2E), instrumentarii si monitorizarii, procesului de Continuous Integration (CI)/Continuous Deployment (CD), infrastructurii, iar in cazul problemelor a debugging-ului si a analizei cauzelor principale (Root Cause Analysis (RCA)).

Un inginer software care lucreaza intr-o companie ce serveste milioane de clienti din intreaga lume trebuie sa fie la curent cu cele mai noi practici din domeniu, sa stie notiuni de programare, securitate cibernetica, networking si infrastructura, dar sa si retina simultan informatii vitale despre arhitectura si sistemele companiei, totul pe langa instruirea colegilor noi sau sedinte.

Timpul unui inginer este astfel impartit de foarte multe procese dinamice care trebuiesc adresate de un angajat cu experienta.

Avand in vedere cele de mai sus, cat si evolutia recenta a sistemelor bazate pe Inteligență Artificială (IA) ce folosesc un Large Language Model (LLM) pentru intelegerea si generarea de text, era natural ca directia in care dezvoltarea software urma sa se schimbe catre un proces in care multe din sarcinile pe care un inginer le-ar fi avut sunt acum delegate sistemelor automate.

Datorita faptului ca aceste modele functioneaza cu text ca date de intrare iar programarea este realizata de asemenea folosind text, nu a fost dificil sa se treaca de la o experienta pur conversationala cu aceste modele la intrebuintari pentru cautare prin limbaj natural, iar in final integrandu-se cu mediile de dezvoltare software si cu terminalul sistemului de operare pentru a avea acces la un context mai bun dar si pentru executia de comenzi putand astfel ca un model de IA sa interactioneze cu mediul inconjurator.

SCOPUL SI OBIECTIVELE LUCRARI

Scopul acestei lucrari este de a realiza un asistent extensibil bazat pe IA, agnostic fata de platforma, care poate functiona cu o multitudine de modele de tipul LLM sau Small Language Model (SLM), ce ruleaza local sau in cloud si ce poate interactiona cu mediul inconjurator (sistem de operare, mediu de dezvoltare, internet) prin intermediul uneltelor (tool-uri ¹), realizat de un workflow agentic ²

Obiectivul principal care trebuie atins este realizarea unui produs functional ce poate ajuta un inginer software in cat mai multe aspecte ale meseriei sale. Pentru a reusi sa indeplinim acest obiectiv, trebuie sa avem in vedere urmatoarele necesitati: interfata facila ce poate fi folosita

¹In IA folosita cu ajutorul agentilor **tool**-urile sunt functii cu descrieri detaliate de folosire pe care agentul le populeaza cu parametrii, le executa si receptioneaza raspunsul pentru a-l folosi mai departe in rezolvarea problemelor complexe

²**Agentii** sunt aplicatii ce folosesc inteligenta artificiala pentru a interactiona cu mediul inconjurator

oriunde folosind terminalul sistemului de operare (Terminal User Interface (TUI)), suficiente unelte pentru a indeplini sarcini cat mai complexe, management-ul conversatiei, contextului si al memoriei, versionare, evaluari si teste de regresie, monitorizare si observabilitate (metrici), siguranta (code execution sandboxes pentru executia de cod, guardrails, Out of Scope (OOS) detection), si cel mai important: un system prompt ales cu grija.

Doua obiective pe care aceasta lucrare nu le prezinta sunt acceptarea de date de intrare multimodale (imagini, documente, videoclipuri) si punerea in piata a acestui produs.

Capitolul 1

LUCRARI ANTERIOARE

O buna intelegere a tehnologiilor, standardelor si a obstacolelor actuale din industrie necesita studierea literaturii de specialitate, a lucrarilor si a arhitecturilor existente deja in domeniu. Acest pas este obligatoriu pentru alegeri informate in cadrul implementarii proiectului, evitarea problemelor deja recunoscute si confera o vedere de ansamblu asupra stadiului final al lucrarii.

1.1 Drumul catre modelele mari de limbaj (LLM)

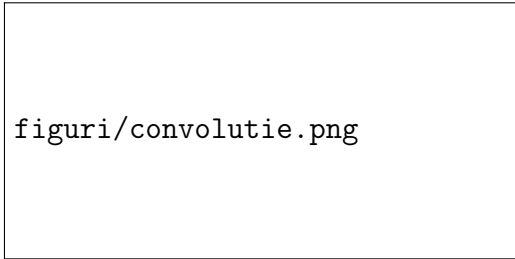
Domeniul inteligentei artificiale a fost studiat o perioada indelungata iar cateva publicatii au marcat industria prin concepte revolutionare si progrese importante.

Unul din conceptele de baza ale inteligentei artificiale a aparut pentru prima data publicat in anul 1958 de catre F. Rosenblatt in [3], unde ni se prezenta perceptronul: o replica a unui neuron biologic capabil sa invete clasificare binara. Modelul invata prin ajustarea ponderilor proprii in functie de eroarea produsa la iesire, formand baza invatarii artificiale. Desi revolutionar, aceasta abordare restrictiona la o singura dimensiune datele de intrare.



Figura 1.1: Modelul unui perceptron

Schimbarea a venit cand Yann LeCun si echipa sa au introdus in [4] retelele neurale convolutive (Convolutional Neural Network (CNN)), un concept care planuia sa rezolve aceasta problema. Au aparut astfel primele aplicatii ale inteligentei artificiale in domeniul imaginilor. Ideea de baza era simpla: in loc de datele unidimensionale caracteristice perceptronului, acum modelul folosea "ferestre" ale unei imagini, mai multe straturi cat si operatii speciale pentru a invata caracteristici spatiale complexe.



figuri/convolutie.png

Figura 1.2: Convolutia cu un nucleu a unei imagini

Deși CNN sunt foarte bune în a procesa date statice, alte aplicații precum procesarea limbajului nu pot fi încadrate într-o "fereastră". Vorbirea și textul sunt secvențiale, deci se întâmplă în timp. Încercările inițiale în a înfrunta această problemă se foloseau folosind Recurrent Neural Network (RNN) se loveau de o problemă fundamentală a învățării artificiale: dispariția gradientilor. Datorită conceptului de Backpropagation Through Time (BTT), în cadrul unor secvențe foarte lungi, gradientii deveneau atât de mici încât ajungeau să fie irelevanți, cauzând modelul să uite date de la începutul secvenței. Progresul a fost atins când Hochreiter and Schmidhuber au introdus conceptul de Long Short Term Memory (LSTM) în [5] care a dat inteligenței artificiale o memorie funcțională. Prin conceptul de "porti", rețeaua reușea să controleze ce informație trebuie să fie uitată și care trebuie să fie păstrată.



Figura 1.3: RNN și LSTM comparate

Următorul mare pas pentru industria urma să fie arhitectura transformer prezentată de Vaswani et al. în celebra lucrare [6]. Arhitectura LSTM trebuia să proceseze pas cu pas o secvență, iar această limitare făcea antrenarea lor pe o cantitate mare de date foarte ineficientă. Arhitectura transformer utilizează o tehnică numită "self-attention" pentru a se uita la fiecare cuvânt dintr-o secvență simultan, calculând astfel legătura dintre acestea. Odată dispărută limitarea din punctul de vedere al vitezei de antrenare, s-a deschis poarta antrenării pe baze de date uriașe, lucru care a dat naștere modelelor mari de limbaj.

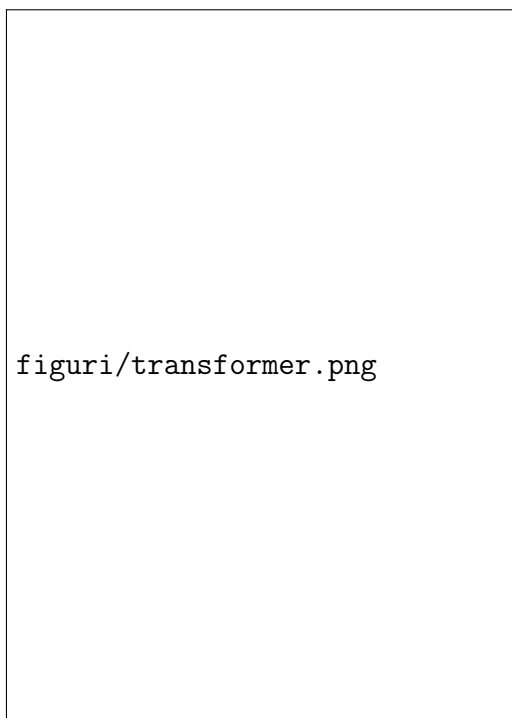


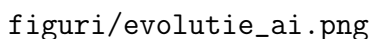
Figura 1.4: Arhitectura transformer

Modelele mari de limbaj sunt produsul antrenării pe cantități masive de date sustrate din întreg internetul. Văzând că dacă continuau să introducă și mai multe date de antrenare, cercetătorii de la OpenAI performanța modelului nu părea să scadă, au reușit să introducă 175 de miliarde de parametri în modelul GPT-3, iar descoperirile au fost relatate în lucrarea "Language Models are Few-Shot Learners" [7]. Marea realizare a fost că fără o instruire prealabilă modelul a învățat singur cum să rezolve sarcini de programare și sumarizare a textului, poezie dar și răspunsuri punctuale la întrebări. Inteligența artificială a făcut pasul de la întrebări specializate la proficiență într-o multitudine de domenii.

1.2 Maturizarea modelelor mari de limbaj și cercetări actuale

Odată cu apariția primelor foundation models ¹ precum GPT-3 sau GPT-4, Sonnet sau Opus, industria inteligenței artificiale a explodat. Aplicații de toate tipurile au început să folosească aceste modele pentru diverse sarcini: programare, traduceri sau experiențe conversaționale. Odată cu cererea, companiile au investit masiv în cercetare și centre ce pot antrena modele mai puternice. Astfel, în fiecare an au apărut modele din ce în ce mai performante. Marii jucători în domeniul acestor modele, printre care și OpenAI, Anthropic, Google și Meta se întrec în a scoate cel mai bun model.

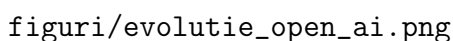
¹Un foundation model este un model mare de limbaj de uz general antrenat pe cantități foarte mari de date diverse, ce poate fi adaptat pentru un număr mare de întrebări.



figuri/evolutie_ai.png

Figura 1.5: Evolutia modelelor in anii recenti [1]

Multe dintre evolutiile facute in anii recenti au fost facute in capabilitatea de a gandi mai adanc, marimea contextului, pre-antrenarea pe anumite date si integrarea cu aplicatii sau Application Programming Interface (API)-uri terte. Un exemplu de evolutie al unui model foundational poate fi regasit in figura de mai jos:



figuri/evolutie_open_ai.png

Figura 1.6: Evolutia modelelor GPT [1]

1.3 Solutii agentice pentru industria dezvoltarii software

Arhitecturile agentice presupun inzestrarea modelelor mari de limbaj cu uneltele necesare interactionarii cu ecosistemul in care ruleaza. Odata cu implementarea procedului "Chain of Thought" [8] care permite navigarea problemelor complexe de catre un model de limbaj, generarea de planuri si cu abordarea ReAct [9] care impreuna incearca sa combata fenomenul halucinatiilor ² si inconsistenta in raspunsuri cu care se confruntau primele modele mari de limbaj.

Increderea in procesul de gandire al modelelor de inteligenta artificiala s-a cimentat, incepand sa produca rezultate consistente, iar astfel a urmat inzestrarea acestora cu unelte care sa le permita interactiunea cu mediul in care ruleaza. Numarul uneltelor ce le ofera capabilitatea de a cauta pe internet pentru a gasi date actuale si a nu se limita la datele de antrenare, la posibilitatea rularii de comenzi in terminalul sistemului de operare sau chiar preluarea controlului browserului personal pentru automatizari, a crescut exponential, si odata cu el si numarul de aplicatii ale acestor modele.

Industria dezvoltarii software are probabil una din cele mai profitabile intrebutari datorita volumului masiv de munca existent in domeniu, asa ca nu a durat mult pana ce marii jucatori pe piata modelelor de IA au inceput sa ofere produse care sa asiste programatorii in sarcinile lor. La momentul actual exista mai multe solutii competitive: Codex de la OpenAI, Gemini CLI de la Google sau Claude Claude de la Anthropic. Toate acestea sunt solutii proprietare care sunt facute sa ruleze cu modelele proprii si pentru care utilizatorul trebuie sa plateasca. Exista si solutii open-source precum OpenCode, Kiro sau OpenClaw cu care detinatorii de modele se lupta pentru a domina piata, desi lupta nu este stransa din cauza monopolului solutiilor proprietare.

²Fenomenul de halucinatii in modelele de limbaj se refera la generarea unor raspunsuri care nu sunt ancorate in fapte reale sau care nu au legatura cu prompt-ul de intrare

Companie	Model	Pret [\$/MTok]	Capabilitati
Anthropic (link)	Claude Opus 4.6	\$5/input MTok, \$25/output MTok	Lent, cel mai inteligent, gandire adaptiva, 1M fereasta de context, 128k tokens max. la iesire
	Claude Sonnet 4.6	\$3/input MTok, \$15/output MTok	Mai rapid, compromite niste inteligenta pentru performante, gandire adaptiva, 1M fereasta de context, 64k tokens max. la iesire
	Claude Haiku	\$1/input MTok, \$5/output MTok	Cel mai rapid, fara gandire adaptiva, 200k fereasta de context, 64k tokens max. la iesire
OpenAI (link)	gpt-5.4	\$2.5/input MTok, \$15/output MTok	Cel mai inteligent pentru sarcini agentice, de programare si profesionale. 1M fereasta de context si 128k tokens max. la iesire
	gpt-5.4-mini	\$0.75/input MTok, \$4.5/output MTok	Model mai mic, rapid dar puternic pentru programare si subagenti. 400k fereasta de context si 128k tokens max. la iesire
	gpt-5.4-nano	\$0.20/input MTok, \$1.25/output MTok	Model mic si foarte rapid pentru sarcini usoare in volum mare. 200k fereasta de context si 128k tokens max. la iesire
Google (link)	Gemini 3.1 Pro	\$2/input MTok, \$12/output MTok < 200k tokens, \$4/input MTok, \$118/output MTok ≥ 200k tokens	Model pentru sarcini complexe ce necesita cunostiinte dezvoltate. Fereasta de context 1M in / 64k out
	Gemini 3.1 Flash	\$0.5/input MTok, \$3/output MTok	Inteligenta comparabila cu modelul Pro la o viteza mai mare. Fereasta de context 1M in / 64k out
	Gemini 3.1 Flash-Lite	\$0.25/input MTok, \$1.5/output MTok	Model construit pentru costuri reduse si volum mare de sarcini. Fereasta de context 1M in / 64k out
DeepSeek (link)	Deepseek Chat 3.2	\$0.028 sau \$0.28/input MTok, \$0.42/output MTok	Mod fara gandire adanca, 128k fereasta de context, max. out token: 8k
	Deepseek Reasoner 3.2	\$0.028 sau \$0.28/input MTok, \$0.42/output MTok	Mod cu gandire adanca, 128k fereasta de context, max. out token: 64k
Meta (link)	Llama 4	Open Source	Performante similare cu GPT4-o sau Gemini 2.0 Flash, dar open-source. Fereasta de context 10M/1M in functie de model.

Tabela 1.1: Comparatia modelelor existente si a capabilitatilor acestora

Nota: Toate modelele au si versiuni pentru diferite nivele de efort pentru gandire (reasoning effort) sau pentru date multimodale ce pot influenta si costul dar si rezultatele

Din punct de vedere al puterii de rezolvare ale sarcinilor reale din industria dezvoltarii software, clasamentul celor mai bune modele se schimba frecvent. Utilizand SWE-Bench Pro [2], putem masura puterea de abordare a acestor modele de top contra unor probleme reale.

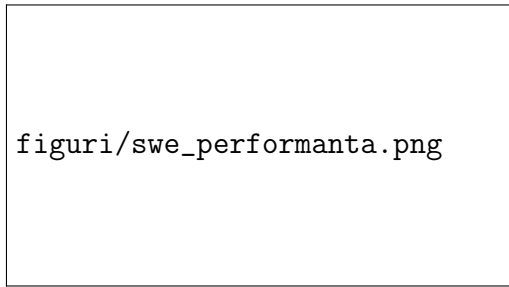


Figura 1.7: Clasamentul modelelor in SWE-Bench Pro [2] la momentul actual in functie de performanta rezolvarii sarcinilor complexe in limba engleza

Pe langa puterea de rezolvare, un alt fenomen a aparut care schimba rata de adoptie a unui model: costul. Desi performantele modelelor cresc si acestea sunt din ce in ce mai integrate in vietile noastre de zi cu zi, asemenea telefoanelor mobile, costurile de rulare ale acestor modele creste si el. Cele mai puternice modele au nevoie de foarte multe resurse pentru antrenare dar si pentru inferenta. Acest lucru se reflecta asupra utilizatorului final care trebuie sa plateasca per-token ³ sau un abonament lunar care include un numar limitat de tokens. Modelele cu capacitati de gandire adanca, pe langa faptul ca au un cost mai mare per-token, consuma si mai multi tokeni din cauza gandirii dezvoltate sau a ferestrei de context mari, ridicand si mai mult costurile. Pentru asta, fiecare detinator de modele fundamentale, are mai multe versiuni ale aceluiasi model, pentru intrebuintari de diverse complexitati, pentru a ajuta cu costurile dar si cu balansarea resurselor. Spre exemplu Google detine modelele din gama Gemini, cu versiunile "Flash", "Thinking", "Pro".

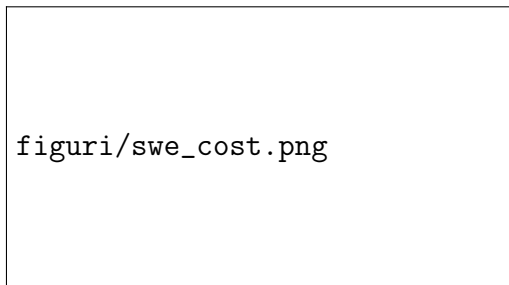


Figura 1.8: Clasamentul modelelor in SWE-Bench Pro [2] la momentul actual in functie de cost

1.4 Necesitati pentru dezvoltarea unei solutii agentice

1.5 Puncte slabe ale arhitecturilor agentice

1.6 Alegerea solutiei de implementare pe baza lucrarilor studiate

³Token-ul este un cuvânt sau o parte din cuvânt și este elementul fundamental procesat de modelele mari de limbaj

Capitolul 2

IMPLEMENTARE

2.1 Inceput

2.2 Proces

2.3 sys prompt

2.4 Provocari

2.5 Solutii

BIBLIOGRAFIE

- [1] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A Survey of Large Language Models, March 2026.
- [2] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?, November 2025.
- [3] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 1958.
- [4] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, November 1998.
- [5] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9:1735–1780, November 1997.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners, July 2020.
- [8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35:24824–24837, December 2022.
- [9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*, September 2022.